

DAY 1 | Monday

Week 1 (Q1) — Repository & Environment

OBJECTIVE: Clean Git repo, Python venv, FastAPI installed, Hello World running.

■ TECHNOLOGY & COMMANDS

```
$ git init && git checkout -b main
$ python3 -m venv venv && source venv/bin/activate
$ pip install fastapi uvicorn[standard] python-dotenv
$ pip freeze > requirements.txt
$ curl -s https://www.gitignore.io/api/python,linux,macos > .gitignore
```

FILES TO CREATE/EDIT:

- .env.example (API_KEY=, DB_URL=)
- main.py (app = FastAPI())
- README.md (Project Mission Statement)

■ ARCHITECTURE & LOGIC

- Monorepo Structure: /apps/api, /apps/web, /libs/core
- Environment Variable Strategy: 12-Factor App
- Pre-commit hooks setup (Black/Isort/Flake8)

■ AUTOPILOT & AI PROMPTS

- Initialize "Coding Agent" instructions in .cursorrules
- Prompt: "You are a Senior Python Backend Engineer. Always type hint."
- Setup "Memory" file for the AI to track decisions.

■ RISK & VALIDATION

RISK: Polluting global python environment. Mitigation: Enforce venv activation.

SUCCESS METRIC: GET / returns {"status": "ok"}

DAY 2 | Tuesday

Week 1 (Q1) — Database Schema & SQLAlchemy

OBJECTIVE: PostgreSQL container running, Models defined, Alembic initialized.

■ TECHNOLOGY & COMMANDS

```
$ docker run -d --name apex-db -p 5432:5432 -e POSTGRES_PASSWORD=secret postgres:15
$ pip install sqlalchemy alembic psycopg2-binary
$ alembic init alembic
```

FILES TO CREATE/EDIT:

- apps/api/database.py (SessionLocal, Base)
- apps/api/models/trade.py (Trade Model)
- apps/api/models/ticker.py (Ticker Model)
- alembic.ini (Target metadata configured)

■ ARCHITECTURE & LOGIC

- ORM Pattern: SQLAlchemy 2.0 (AsyncIO support?)
- Migration Strategy: Alembic auto-generate
- Schema Design: TimescaleDB extension for time-series data?

■ AUTOPILOT & AI PROMPTS

- Prompt: "Generate SQLAlchemy model for Trade with fields: symbol, side, qty, price, timestamp."
- Ask AI to write the `__repr__` methods for debugging.

■ RISK & VALIDATION

RISK: Schema drift. Mitigation: CI check for missing migrations.

SUCCESS METRIC: alembic upgrade head runs successfully.

DAY 3 | Wednesday

Week 1 (Q1) — API CRUD & Pydantic

OBJECTIVE: Create/Read Trades via HTTP endpoints with Validation.

■ TECHNOLOGY & COMMANDS

```
$ pip install pydantic
$ uvicorn apps.api.main:app --reload
```

FILES TO CREATE/EDIT:

- apps/api/schemas/trade.py (TradeCreate, TradeResponse)
- apps/api/routers/trades.py (@router.post)
- apps/api/main.py (include_router)

■ ARCHITECTURE & LOGIC

- DTO Pattern: Pydantic models separate from ORM models.
- Dependency Injection: get_db() session yield.
- HTTP Status Codes: 201 Created, 404 Not Found.

■ AUTOPILOT & AI PROMPTS

- Prompt: "Write Pydantic schema for Trade with validation (qty > 0)."
- Prompt: "Create FastAPI endpoint to list trades with pagination."

■ RISK & VALIDATION

RISK: SQL Injection. Mitigation: ORM handles parameterization.

SUCCESS METRIC: Swagger UI (/docs) shows Create Trade endpoint.

DAY 4 | Thursday

Week 1 (Q1) — Testing & CI/CD

OBJECTIVE: Pytest suite running locally and in GitHub Actions.

■ TECHNOLOGY & COMMANDS

```
$ pip install pytest httpx pytest-asyncio
$ pytest tests/ -v
```

FILES TO CREATE/EDIT:

- tests/conftest.py (Test DB fixture)
- tests/test_trades.py (test_create_trade)
- .github/workflows/test.yml

■ ARCHITECTURE & LOGIC

- Testing Pyramid: Unit vs Integration.
- Fixture Factory Pattern.
- CI/CD: Fail build on lint error.

■ AUTOPILOT & AI PROMPTS

- Prompt: "Write a test that mocks the specialized database session."
- Prompt: "Generate GitHub Action YAML for running pytest on PR."

■ RISK & VALIDATION

RISK: Flaky tests. Mitigation: Deterministic seed data.

SUCCESS METRIC: GitHub Action turns Green.

DAY 5 | Friday

Week 1 (Q1) — Frontend Shell (React/Vite)

OBJECTIVE: Typescript React app running, fetching data from API.

■ TECHNOLOGY & COMMANDS

```
$ npm create vite@latest apps/web -- --template react-ts
$ cd apps/web && npm install
$ npm install @tanstack/react-query axios tailwindcss
```

FILES TO CREATE/EDIT:

- apps/web/src/api/client.ts (Axios instance)
- apps/web/src/hooks/useTrades.ts (React Query hook)
- apps/web/src/App.tsx (Render list of trades)

■ ARCHITECTURE & LOGIC

- State Management: React Query (Server State) + Zustand (Client State).
- Styling: Tailwind CSS Utility-first.
- CORS config on FastAPI backend.

■ AUTOPILOT & AI PROMPTS

- Prompt: "Create a React component that displays a table of trades."
- Prompt: "Configure Tailwind to use Slate-900 dark mode default."

■ RISK & VALIDATION

RISK: CORS errors. Mitigation: allow_origins=["*"] in dev.

SUCCESS METRIC: Frontend displays "Hello World" data from Backend.

DAY 6 | Monday

Week 2 (Q1) — Market Data Feed (Alpaca/Polygon)

OBJECTIVE: Streaming Real-Time Bars for AAPL into console.

■ TECHNOLOGY & COMMANDS

```
$ pip install alpaca-py
$ python scripts/test_stream.py
```

FILES TO CREATE/EDIT:

- apps/api/services/market_data.py
- scripts/stream_quotes.py

■ ARCHITECTURE & LOGIC

- WebSocket Client pattern (asyncio).
- Data Normalization (Bar object).
- Queue System (Producer -> Consumer).

■ AUTOPILOT & AI PROMPTS

- Prompt: "Write an asyncio wrapper for Alpaca WebSocket."
- Prompt: "Handle reconnection logic with exponential backoff."

■ RISK & VALIDATION

RISK: API Rate limits. Mitigation: Check headers.

SUCCESS METRIC: See live price updates in terminal.

DAY 7 | Tuesday

Week 2 (Q1) — Broker Interface (Paper Trading)

OBJECTIVE: Place a Market Buy Order via Code.

■ TECHNOLOGY & COMMANDS

```
$ pip install pytz
$ python scripts/sub_order.py
```

FILES TO CREATE/EDIT:

- apps/api/services/broker.py (AbstractBaseClass)
- apps/api/services/alpaca_broker.py (Implementation)

■ ARCHITECTURE & LOGIC

- Interface Segregation Principle.
- Order Lifecycle (Pending -> New -> Filled).
- Idempotency Keys.

■ AUTOPILOT & AI PROMPTS

- Prompt: "Create a Broker abstract class with buy_market method."
- Prompt: "Implement cancel_all_orders logic."

■ RISK & VALIDATION

RISK: Accidental Real Money trade. Mitigation: Check PAPER_MODE env var.

SUCCESS METRIC: Order ID returned from Alpaca Paper API.